

RAD (Rapid Application Development) Model

The **RAD Model** is an **incremental software process model** designed for **high-speed development**. It adopts a **component-based approach** similar to the **Waterfall model** but accelerates the process, allowing the creation of a fully functional system within **60 to 90 days**.

Key Features of RAD

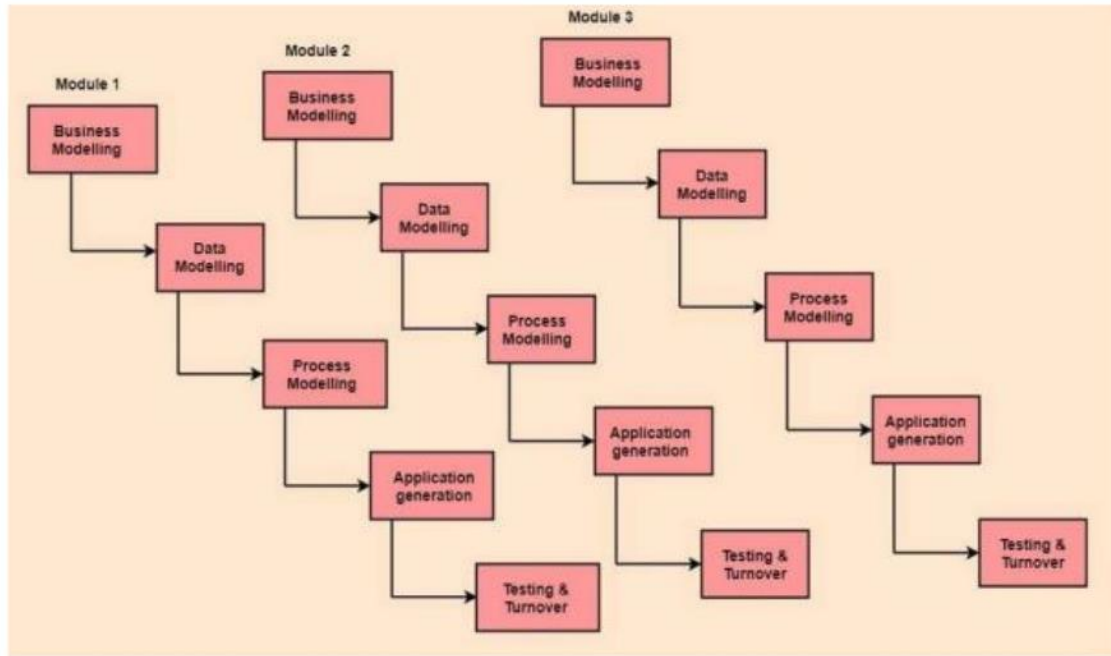
- ✓ **Incremental:** Develops the product in small, manageable parts (increments).
- ✓ **Short Development Cycle:** Focus on rapid completion of activities.
- ✓ **Component-Based:** Reuses existing components to speed up development.
- ✓ **Parallel Development:** Multiple teams work on different parts of the project simultaneously.

Phases of the RAD Model

1. **Communication Phase**
 - a. Discuss requirements with stakeholders.
 - b. Define clear business needs and expectations.
2. **Planning Phase**
 - a. Break down the project into modules or components.
 - b. Allocate tasks to different development teams.
3. **Modeling Phase** (Three sub-phases)
 - a. **Business Modeling** – Identify business processes and information flow.
 - b. **Data Modeling** – Define data objects and relationships.
 - c. **Process Modeling** – Define how data is processed and transformed.
4. **Construction Phase**
 - a. Develop using **reusable components**.
 - b. Automate code generation where possible.
 - c. Perform quick testing alongside development.
5. **Deployment Phase**
 - a. Integrate different modules.

- b. Deliver a working system to the customer.
- c. Get customer feedback and refine if needed.

THE RAD MODEL



⚠ Challenges of the RAD Model

- ✗ **Needs Multiple Teams:** Parallel development requires several skilled teams working together.
- ✗ **Requires Customer Involvement:** Constant feedback is essential for success.
- ✗ **High Dependency on Modularity:** System must be designed in independent modules.
- ✗ **Not Suitable for High-Risk Projects:** If technical challenges are significant, RAD can fail due to the fast pace.

🚀 CMMI (Capability Maturity Model Integration)

CMMI is a **proven industry framework** designed to improve **product quality** and **development efficiency** for both **hardware and software**. It helps organizations streamline their processes, reduce risks, and improve business outcomes.

☑ Key Features of CMMI

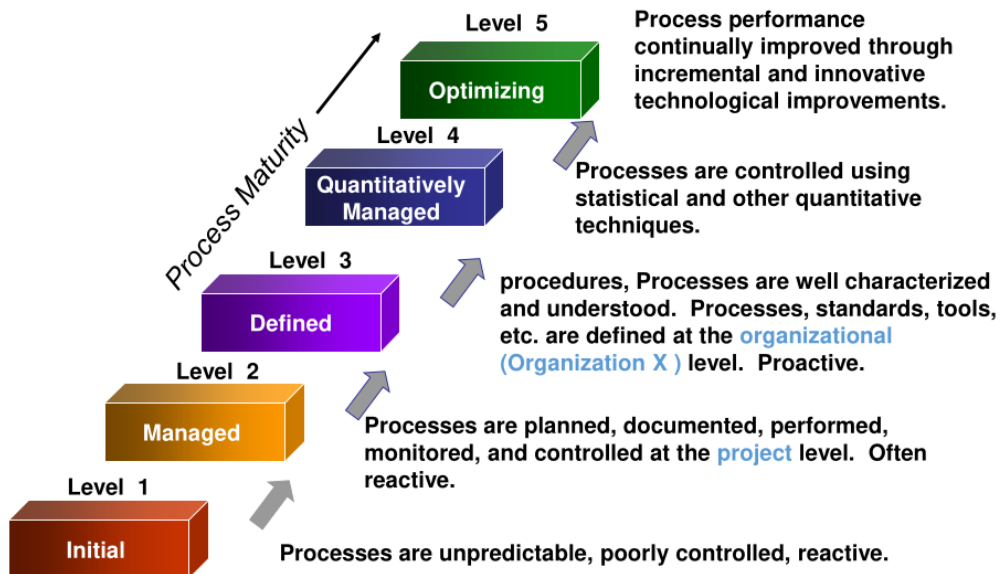
- ✓ **Industry-Proven:** Developed with input from companies like Motorola and Ericsson.
- ✓ **Focus on Business Outcomes:** Aimed at improving overall business efficiency.
- ✓ **Structured Approach:** Uses defined levels to measure the maturity of an organization's processes.
- ✓ **Applicable to Hardware and Software:** Covers both technical and managerial aspects.

📖 CMMI Framework Components

1. **Models** – CMMI includes multiple models depending on the development area:
 - a. **SE/SW:** Systems Engineering + Software Engineering
 - b. **SE/SW + IPPD:** Systems Engineering + Software Engineering + Integrated Product and Process Development
 - c. **SE/SW + IPPD + SS:** Adds Supplier Sourcing to the above
 - d. **Software Engineering Only:** Focus on software processes
2. **Representation Options** – Two approaches to implement CMMI:
 - a. **Staged:** Follows a defined progression of maturity levels (recommended for consistency).
 - b. **Continuous:** Improves specific processes based on business priorities (more flexible).
3. **Definition of Systems Engineering** –

"An interdisciplinary approach governing the total technical and managerial effort required to transform customer needs, expectations, and constraints into a product solution and support it throughout the product's life."

CMMI Staged Representation - 5 Maturity Levels



✿ 5 Maturity Levels of CMCI (Staged Representation)

- 1. Initial (Level 1):**
 - a. Processes are ad-hoc and chaotic.
 - b. No structured approach to development.
- 2. Managed (Level 2):**
 - a. Basic project management practices are established.
 - b. Requirements, schedules, and project tracking are defined.
- 3. Defined (Level 3):**
 - a. Organization-wide standards are established.
 - b. Processes are documented and followed consistently.
- 4. Quantitatively Managed (Level 4):**
 - a. Processes are measured and controlled using data.
 - b. Focus on process improvement using metrics.
- 5. Optimizing (Level 5):**
 - a. Continuous process improvement.
 - b. Focus on innovation and addressing process weaknesses.

When to Use CMMI

- ✓ Ideal for **large organizations** with complex processes.
- ✓ Best suited for **mission-critical projects** (e.g., aerospace, healthcare).
- ✓ Works well when **scalability** and **reliability** are key goals.

Agile Model

The **Agile Model** is one of the most widely used software development models in the digital era. It's an **incremental and iterative** approach that focuses on **flexibility** and **customer satisfaction** by adapting to changing requirements, technologies, and user needs.

Key Features of Agile

- ✓ **Incremental & Iterative:** Develop software in small, workable parts delivered frequently.
- ✓ **Adaptive:** Responds to changing requirements even late in the development process.
- ✓ **Customer-Centric:** Continuous customer involvement ensures the product meets expectations.
- ✓ **Collaborative:** Business stakeholders and developers work closely together.

When to Use the Agile Model

1. **Large Projects:** Ideal for complex and large-scale software development.
2. **Frequent Requirement Changes:** When customer needs and business goals are dynamic.
3. **Flexible Timeline and Budget:** Works well when project delivery dates and costs are not fixed.
4. **Experienced Team:** Success depends on a highly qualified and self-organizing development team.
5. **Continuous Customer Interaction:** Requires frequent feedback from stakeholders.

Agile Principles

1. **Customer Satisfaction:** Highest priority is to deliver functional software early and continuously.
2. **Embrace Change:** Changes are welcomed at any point in development.
3. **Frequent Deliveries:** Deliver working software in short cycles (every 2–4 weeks).
4. **Collaboration:** Strong communication between developers and business stakeholders.
5. **Motivated Teams:** Empower teams with the right environment and support to increase productivity.
6. **Face-to-Face Communication:** Direct interaction ensures quick feedback and better understanding.
7. **Technical Excellence:** Continuous improvement in design and code quality enhances agility.
8. **Sustainable Development:** Maintain a constant pace for long-term efficiency.
9. **Simplicity:** Focus on the essentials; avoid unnecessary work and documentation.
10. **Self-Organizing Teams:** The best designs and solutions come from experienced, independent teams.
11. **Adaptive Architecture:** Design and requirements evolve based on experience.
12. **Regular Reflection:** Teams analyze and adjust their process to improve delivery.

Advantages and Disadvantages of Agile

Advantages	Disadvantages
✓ Strong customer involvement ensures satisfaction.	× Lack of proper documentation makes future maintenance difficult.
✓ High flexibility to changing requirements.	× Heavy dependence on customer involvement — unclear input can mislead the team.
✓ Fast and frequent deliveries keep customers engaged.	× Not ideal for projects with fixed scope and requirements.
✓ Focus on design and development over documentation.	× If the team lacks experience, Agile can lead to poor execution.
✓ Encourages teamwork and direct communication.	× Misunderstanding of Agile principles can cause disorganization.
✓ Reduces overall development time by releasing frequent updates.	× Requires experienced and self-organizing teams.

When Agile Works Best

- ✓ For projects where **requirements are expected to change**.
- ✓ When the team is **skilled and experienced**.
- ✓ When **continuous customer feedback** is available.
- ✓ For startups and **innovative projects** needing quick market entry.

Introduction To Agile Tool: Jira

- - JIRA is a tool developed by Australian Company Atlassian.
- - It is used for bug tracking, issue tracking, and project management.
- - The name "JIRA" is actually inherited from the Japanese word "Gojira" which means "Godzilla".
- - The basic use of this tool is to track issue and bugs related to your software and Mobile apps.
- - JIRA is an incident management tool.
- - JIRA is a platform independent tool; it can be used with any OS.
- - JIRA is multi-lingual tool – English, French, German, Japanese, Spanish, etc.
- - JIRA supports MySQL, Oracle, PostgreSQL and SQL server in the backend.
- - JIRA can be integrated with many other tools – Subversion, GIT, Clear case, and many more

Extreme Programming (XP)

Extreme Programming (XP) is one of the most widely used **Agile** methodologies. It was introduced by **Kent Beck** in the **late 1980s** to improve software quality and responsiveness to changing customer requirements.

Key Features of Extreme Programming (XP)

- ✓ **Focus on Code Quality:** Emphasizes clean code and simple design.
- ✓ **Frequent Feedback:** Continuous communication with customers and team members.
- ✓ **Rapid and Incremental Development:** Delivers working software frequently.
- ✓ **Testing-Driven:** Automated unit tests are written before the actual code (TDD).
- ✓ **Pair Programming:** Two developers work on the same code — one writes, the other reviews.

XP Values (Defined by Kent Beck)

1. Communication

- a. Effective communication between developers and stakeholders.
- b. Open and transparent dialogue to avoid misunderstandings.

2. Simplicity

- a. Design only for current needs — no over-engineering.
- b. Focus on minimal, clean solutions.

3. Feedback

- a. Three sources of feedback:
 - i. **Implemented software** – See how it works.
 - ii. **Customer** – Direct feedback on features.
 - iii. **Team members** – Suggestions for improvement.

4. Courage (Discipline)

- a. Be willing to refactor code, remove unnecessary features, and adapt to changing needs.
- b. Have the discipline to stick to simple designs.

5. Respect

- a. Encourage a positive team environment.
- b. Respect feedback and suggestions from others.

Scrum

Scrum is an Agile software development framework introduced by **Jeff Sutherland** and his team in the **early 1990s**. It focuses on breaking down work into small, manageable units and delivering working software incrementally through **Sprints**.

Key Features of Scrum

- ✓ **Iterative and Incremental:** Develop software in small cycles (Sprints).
- ✓ **Time-Boxed Sprints:** Sprints are typically 1–4 weeks long.
- ✓ **Customer-Centric:** Continuous customer involvement and feedback.
- ✓ **Defined Roles:** Clear responsibilities for team members (Scrum Master, Product Owner, Team).
- ✓ **Transparency:** Real-time tracking of progress and roadblocks.

Scrum Framework Activities

1. **Requirements:** Define customer needs and goals.
2. **Analysis:** Break down requirements into manageable tasks.
3. **Design:** Create a simple design to meet the current sprint's goal.
4. **Evolution:** Continuously improve based on feedback and testing.
5. **Delivery:** Deliver functional increments at the end of each sprint.

Scrum Process Flow

1. **Backlog:**
 - a. A list of prioritized project requirements or features.
 - b. Items can be added or reprioritized at any time.
2. **Sprint Planning:**
 - a. Define the work for the upcoming sprint.
 - b. Estimate effort and assign tasks to team members.
3. **Sprints:**
 - a. Short development cycles (1–4 weeks).
 - b. Develop and test specific features.
4. **Daily Scrum Meetings:**
 - a. Short 15-minute stand-up meetings.
 - b. Three key questions:
 - i. What did you do since the last meeting?
 - ii. What obstacles are you facing?
 - iii. What will you do before the next meeting?
5. **Demo:**
 - a. Present the working product to the customer.
 - b. Get feedback and adjust as needed.
6. **Sprint Review:**
 - a. Evaluate the sprint's outcome and customer satisfaction.
7. **Sprint Retrospective:**
 - a. Discuss what went well and what needs improvement.